

Laboratorio S05 – Mexi Banco con Docker Compose

Nombre: Francisco Javier Pérez Nava

Materia: Sistemas Distribuidos

Grupo: 2

Profesor: Oscar Manuel Ruiz Hurtado

Práctica: S05 – Mexi Banco con Docker Compose

Paso 1. Clonar el proyecto

Evidencia

```
PS C:\WINDOWS\System32> cd "C:\Users\franc\Desktop\Sistemas Distribuidos"
PS C:\Users\franc\Desktop\Sistemas Distribuidos> git clone --branch v05.1 https://github.com/OscarRuiz21/mexi-banco.git
Cloning into 'mexi-banco'...
remote: Enumerating objects: 123, done.
remote: Counting objects: 100% (123/123), done.
remote: Compressing objects: 100% (97/97), done.
remote: Total 123 (delta 23), reused 107 (delta 12), pack-reused 0 (from 0)
Receiving objects: 100% (123/123), 46.71 KiB | 408.00 KiB/s, done.
Resolving deltas: 100% (23/23), done.
warning: refs/tags/v05.1 cd46b8b91a09070f8262c3b8acfb17caf2405cda is not a commit!
Note: switching to '88203161aa31a371823715684a548d154e1516f8'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

PS C:\Users\franc\Desktop\Sistemas Distribuidos> cd mexi-banco
```

Paso 2. Levantar los servicios con Docker Compose

Durante la ejecución, el puerto 8080 de mi computadora ya se encontraba ocupado. Por esta razón cambié únicamente el puerto externo de la aplicación a 8081, manteniendo el puerto 8080 dentro del contenedor.

Después de levantar nuevamente los servicios, comprobé con `docker compose ps` que `app` y `db` estaban funcionando y en estado `healthy`.

Evidencia

```
PS C:\Users\franc\Desktop\Sistemas Distribuidos\mexi-banco> docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED
STATUS             PORTS
mexi-banco-app-1    mexi-banco-app       "java -jar app.jar"    app        About a minute ago
Up About a minute (healthy)  0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
mexi-banco-db-1     postgres:17-alpine   "docker-entrypoint.s..." db         About a minute ago
Up About a minute (healthy)  5432/tcp
```

Paso 3. Probar Mexi Banco con Postman

Explicación

En este paso se pide importar la colección de Postman proporcionada en la práctica y ejecutar sus peticiones en el orden establecido. Debido al cambio realizado en Docker Compose, la variable `baseUrl` de la colección se configuró como `http://localhost:8081`.

Primero se comprobó que la aplicación estuviera disponible mediante la petición de salud. La respuesta esperada es 200 OK y un estado UP.

Después se creó la cuenta origen de Ana con un saldo inicial de \$1,000 y se consultó para verificar que la información se hubiera guardado correctamente.

En la sección de transferencia interna se creó una cuenta destino con \$500 y se transfirieron \$200 desde la cuenta origen. Después de la operación, el saldo esperado de la cuenta origen es \$800 y el de la cuenta destino es \$700. También se consulta la transferencia para comprobar que quedó registrada.

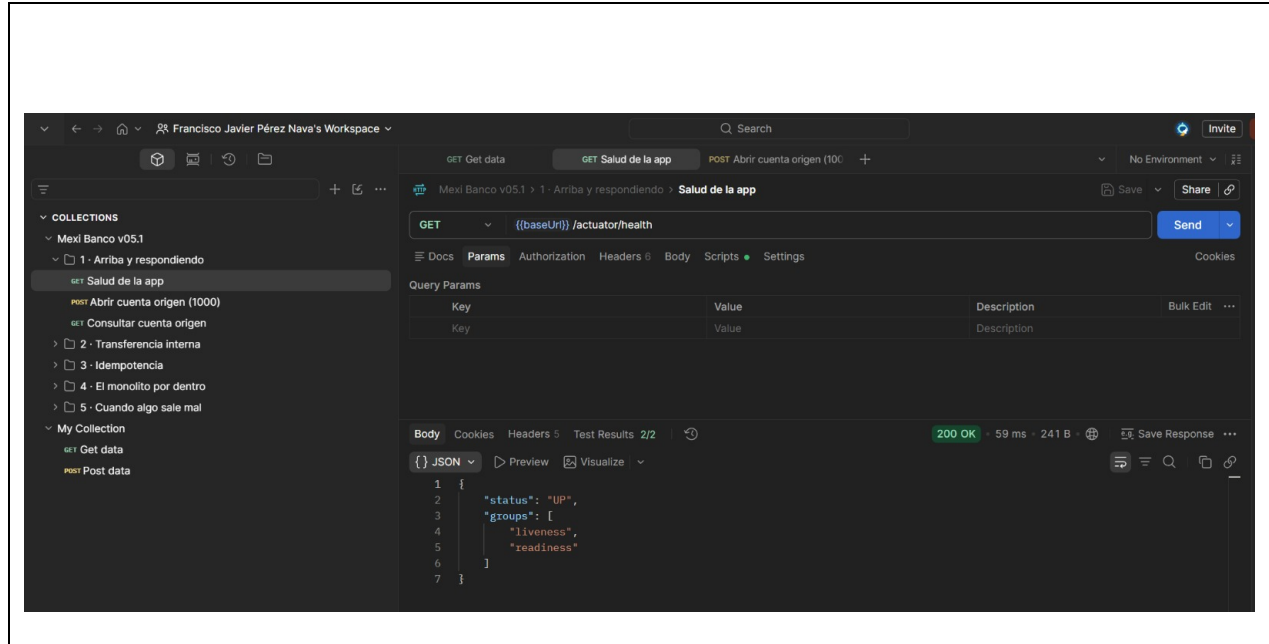
En la prueba de idempotencia se realiza un SPEI de \$50 y posteriormente se vuelve a enviar la misma operación utilizando la misma `Idempotency-Key`. Se debe comprobar que el sistema reconozca el reintento y no realice el cobro dos veces. Por esta razón, después de ambos envíos, la cuenta origen debe quedar con \$750 y no con \$700.

Posteriormente se consulta el estado de cuenta y los avisos generados por las operaciones. Finalmente se ejecutan las pruebas de error incluidas en la colección, como CLABE inexistente, CLABE repetida, cuerpo incompleto, saldo insuficiente, transferencia a la misma cuenta y SPEI sin `Idempotency-Key`. Estas pruebas permiten

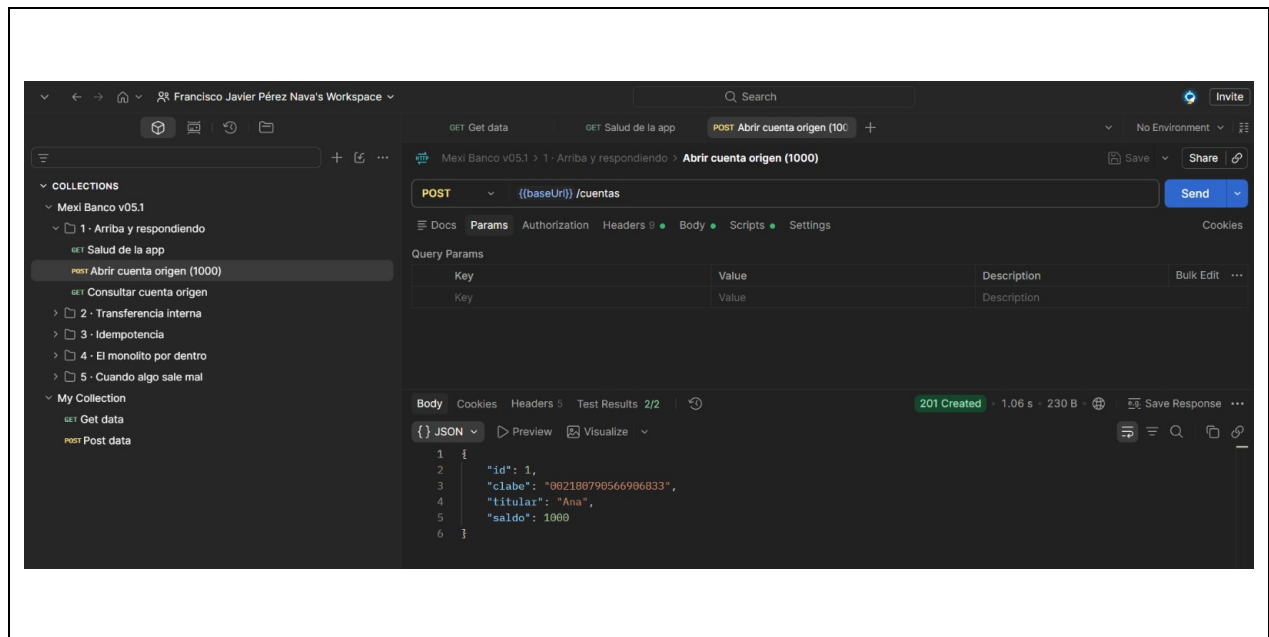
comprobar que la aplicación responde con los códigos de error esperados y que las operaciones inválidas no modifican el saldo.

Evidencias importantes de Postman

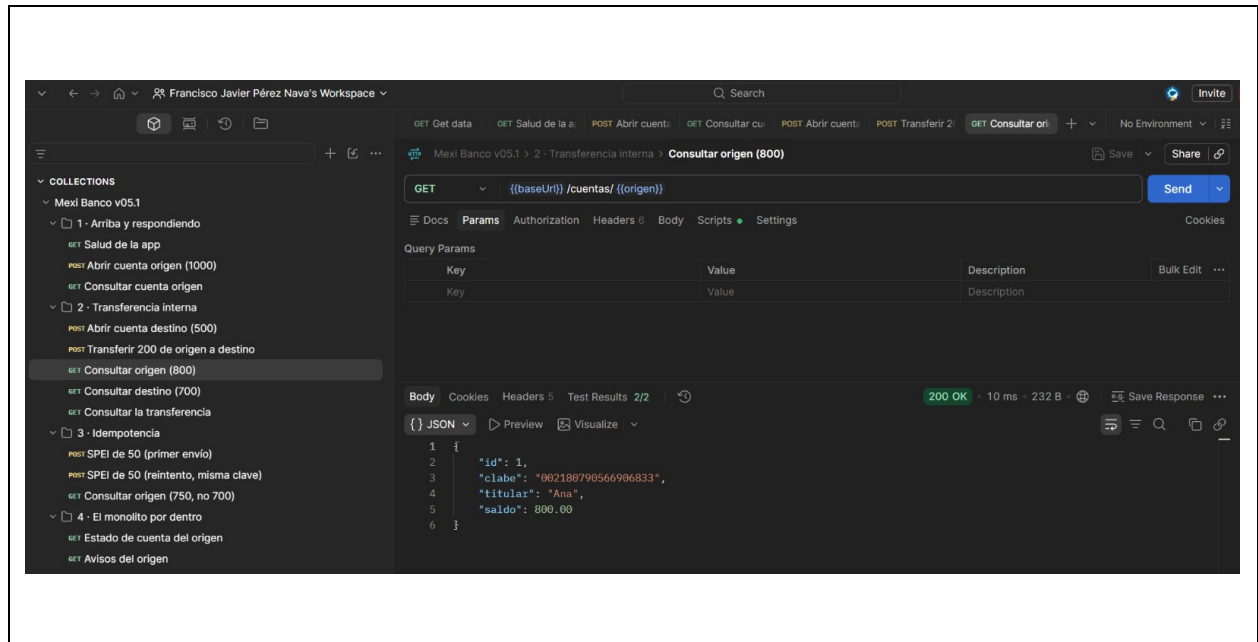
Salud de la aplicación



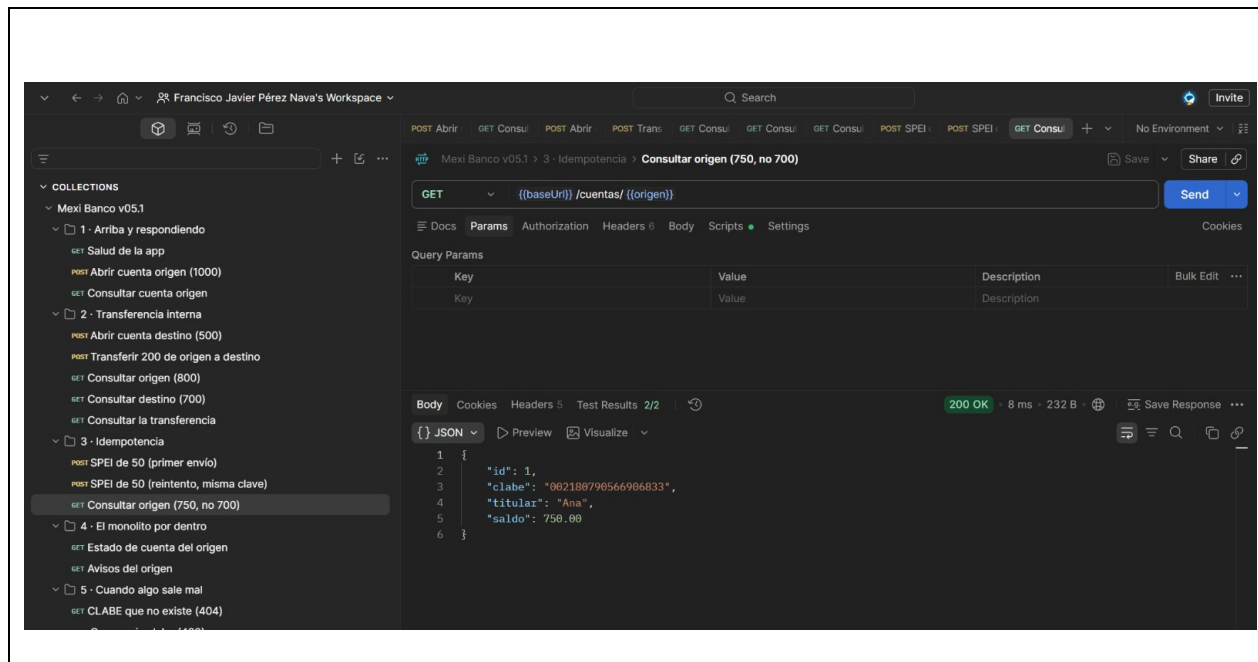
Cuenta origen creada y consultada



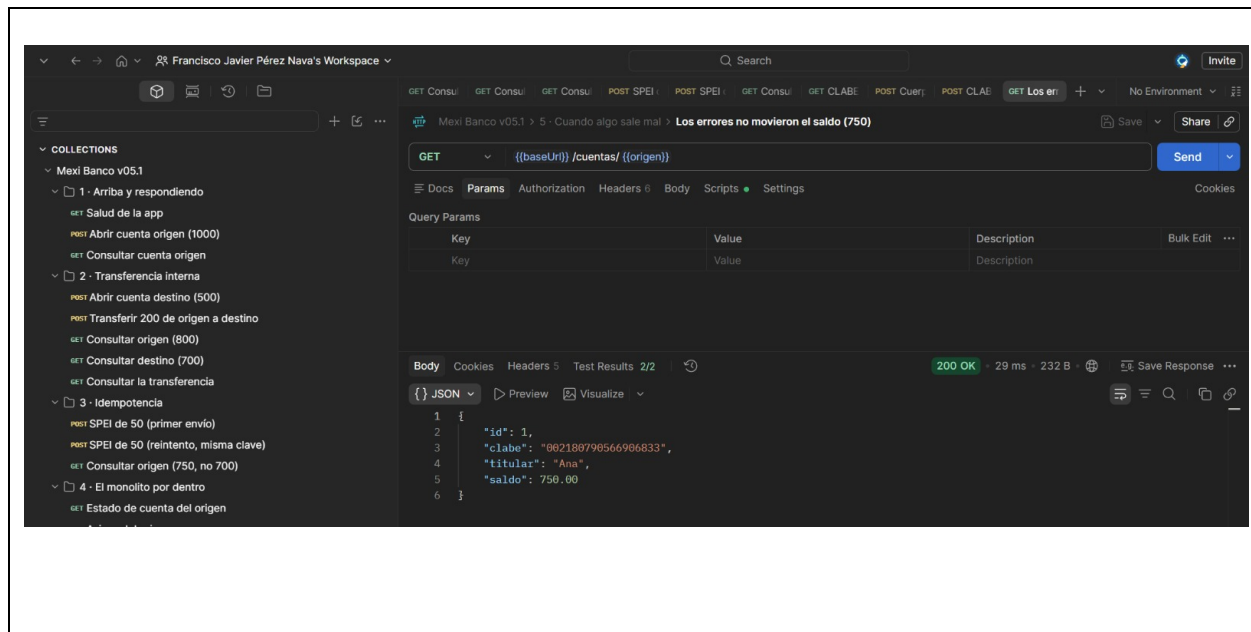
Transferencia interna realizada



Prueba de idempotencia



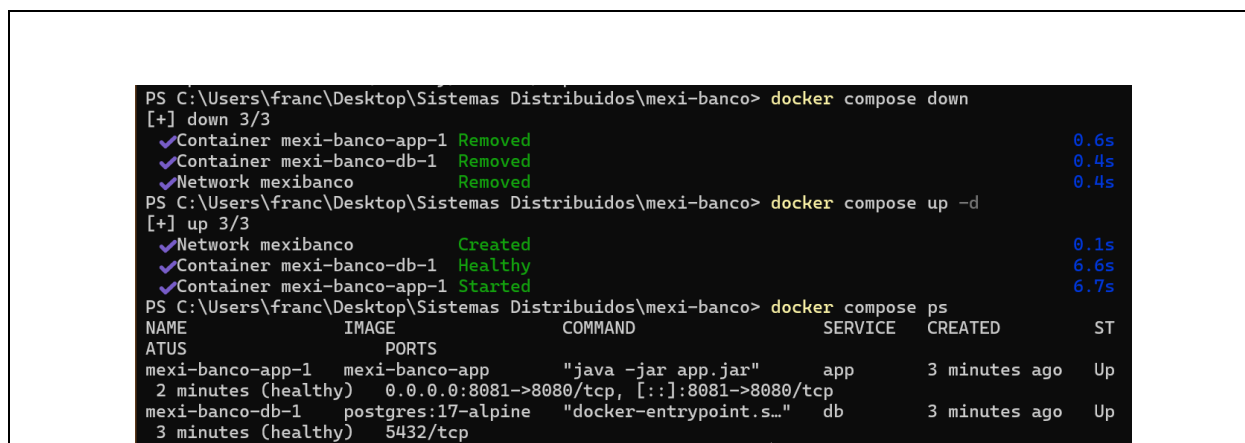
Manejo de errores sin modificar el saldo

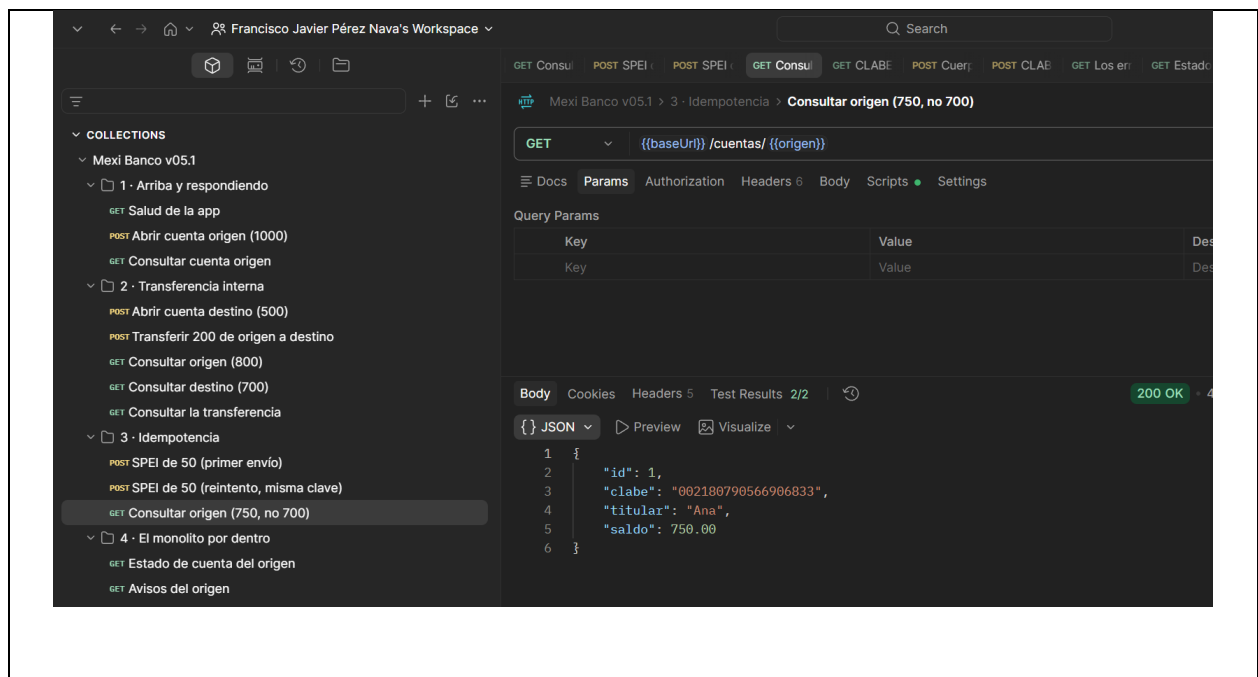


Paso 4. Comprobar la persistencia de los datos

Después de ejecutar `docker compose down` y volver a levantar los servicios, consulté nuevamente la cuenta y los datos seguían disponibles. Esto muestra que eliminar los contenedores no eliminó la información almacenada en el volumen.

Evidencia

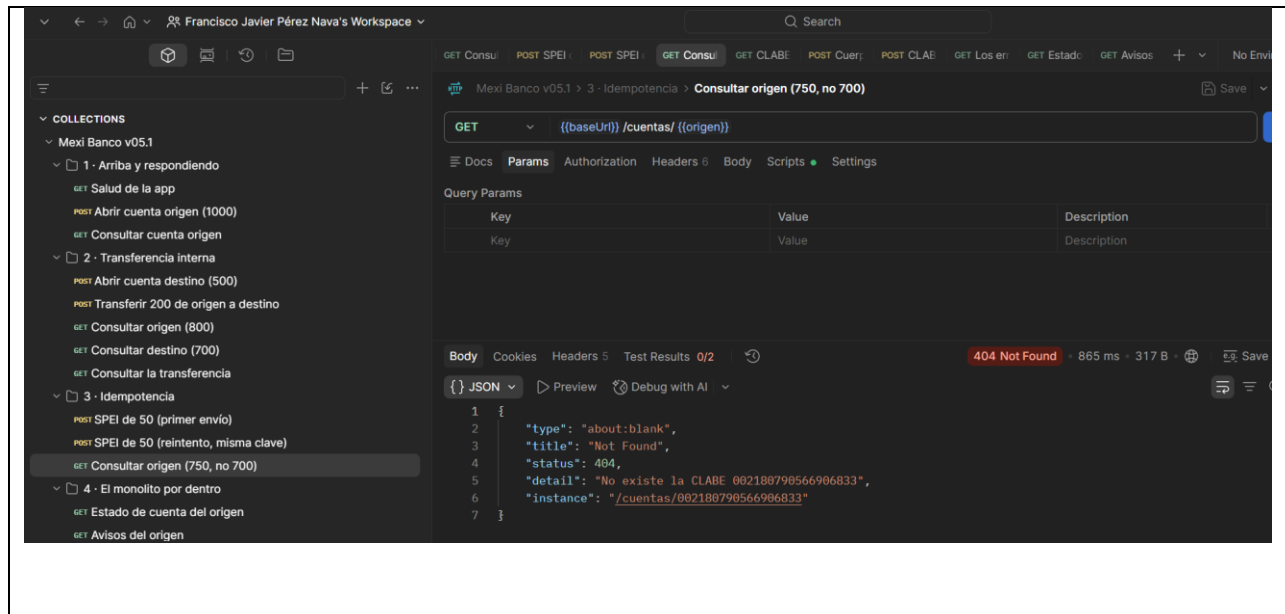




Después de ejecutar `docker compose down -v` y volver a levantar los servicios, la cuenta creada anteriormente ya no se encontraba disponible. La diferencia fue que la opción `-v` también eliminó el volumen donde se almacenaban los datos de la base de datos, por lo que la aplicación inició nuevamente sin la información creada durante las pruebas.

Evidencia

```
PS C:\Users\franc\Desktop\Sistemas Distribuidos\mexi-banco> docker compose down -v
[+] down 4/4
✓Container mexi-banco-app-1      Removed      0.5s
✓Container mexi-banco-db-1      Removed      0.3s
✓Volume mexi-banco_mexibanco_datos Removed      0.1s
✓Network mexibanco              Removed      0.3s
PS C:\Users\franc\Desktop\Sistemas Distribuidos\mexi-banco> docker compose up -d
[+] up 4/4
✓Network mexibanco              Created      0.1s
✓Volume mexi-banco_mexibanco_datos Created      0.0s
✓Container mexi-banco-db-1      Healthy      6.1s
✓Container mexi-banco-app-1      Started      6.3s
PS C:\Users\franc\Desktop\Sistemas Distribuidos\mexi-banco> docker compose ps
NAME                IMAGE                COMMAND                SERVICE  CREATED        ST
ATUS                PORTS
mexi-banco-app-1    mexi-banco-app      "java -jar app.jar"    app      4 minutes ago  Up
4 minutes (healthy) 0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
mexi-banco-db-1     postgres:17-alpine  "docker-entrypoint.s..." db       4 minutes ago  Up
4 minutes (healthy) 5432/tcp
```



Paso 5. Revisar la estructura del monolito

Revisé la estructura del proyecto dentro de `src/main/java/mx/mexibanco`. Encontré los módulos `cuenta`, `movimiento`, `transferencia`, `notificación` y `SPEI`. Dentro de ellos se encuentran componentes como `controller`, `service`, `repository` y `entity`. Que nos permiten separar las diferentes responsabilidades del sistema, aunque todas siguen formando parte de una misma aplicación monolítica.

Paso 6. Revisar Docker Compose y Dockerfile

Explicación

En este paso revisé los archivos `Docker Compose` y `Dockerfile` para entender mejor cómo funciona el proyecto y cómo se prepara para poder ejecutarse, con esto pude ver de una manera más clara cómo se levantan la aplicación y la base de datos y cómo trabajan juntas, además me ayudó a relacionar lo que aparece en estos archivos con los comandos que fui utilizando durante la práctica y así entender mejor qué estaba haciendo Docker en cada momento.